Part 8 — Jenkins Interview Preparation Notes

Complete English Guide | Basic Questions • Scenario-Based Questions • Production Issues • Real-Time Project Explanation

These notes are designed for DevOps Engineer, Cloud Engineer, AWS Support, and Jenkins CI/CD interviews. Each major topic follows your requested five headings.

Topic 1: Jenkins Interview Fundamentals

1. What is Jenkins?

1. Definition — What is it?

Jenkins is an open-source automation server used to automate software development tasks, including building, testing, and deploying applications.

2. Purpose — Why do we use it?

- To automate repetitive manual tasks.
- To implement Continuous Integration (CI) and Continuous Delivery/Deployment (CD).
- To identify code errors early.
- To reduce manual deployment effort.
- To make software releases faster and more consistent.

3. Working — How does it work?

1. A developer pushes code to GitHub.
2. Jenkins detects the change through a webhook or polling.
3. Jenkins checks out the source code.
4. It executes pipeline stages such as build, test, scan, and deployment.
5. It reports whether the pipeline succeeded or failed.

4. Practical Example — How do we use it in a project?

In an AWS Terraform project, Jenkins retrieves Terraform configuration from GitHub, validates the configuration, generates a Terraform plan, and deploys infrastructure to AWS after the required approval.

5. Scenario-Based Interview Questions and Answers

Q: Why did you choose Jenkins instead of manual deployment?

Answer: I used Jenkins to automate the deployment process. Manual deployments take more time and may introduce human errors. Jenkins provides a repeatable pipeline, logs, build history, and controlled deployment stages.

Q: What happens if a Jenkins build fails?

Answer: I first identify the failed stage, review the console output, and determine whether the problem is related to Git, credentials, dependencies, testing, infrastructure, or deployment. I fix the root cause and rerun the pipeline after validating the correction.

Topic 2: Continuous Integration and Continuous Delivery

1. Definition

- Continuous Integration (CI): Developers frequently integrate code changes, and automated processes build and test the code.
- Continuous Delivery (CD): Code is automatically prepared for release, with production deployment typically requiring approval or a controlled release step.
- Continuous Deployment: Every change that passes the required automated checks is automatically deployed to production.

2. Purpose

CI/CD helps teams release software more frequently, detect defects earlier, improve consistency, and reduce manual work.

3. Working

1. **Developer commits code** — Git / GitHub
2. **Jenkins pipeline starts** — Webhook or polling
3. **Build, test and scan** — Automated quality checks
4. **Approval and deployment** — Controlled release to target environment

4. Practical Example

For an application running on Amazon EKS, Jenkins can build a Docker image, run tests, push the image to Amazon ECR, and deploy the image to Kubernetes after approval.

5. Scenario-Based Questions

Q: Your team wants to deploy every code change automatically. What would you implement?

Answer: I would implement a CI/CD pipeline with source checkout, automated testing, security scanning, artifact publishing, and deployment stages. For production, I would also configure appropriate approvals, access controls, monitoring, and rollback procedures based on the team's release policy.

Q: What is the difference between Continuous Delivery and Continuous Deployment?

Answer: Continuous Delivery keeps the application ready for release, generally with a manual or controlled production-release decision. Continuous Deployment automatically releases every change that passes the defined checks.

Topic 3: Jenkins Pipeline and Jenkinsfile

1. Definition

A Jenkins Pipeline is a series of automated steps that define how an application or infrastructure is built, tested, and deployed. A Jenkinsfile stores the pipeline definition as code in the source repository.

2. Purpose

- To version-control pipeline logic.
- To make deployments repeatable.
- To support code reviews and collaboration.
- To separate pipeline stages.
- To simplify troubleshooting and auditing.

3. Working

A pipeline commonly contains:

- pipeline: Defines a Declarative Pipeline.
- agent: Specifies where the pipeline runs.
- stages: Groups the major phases.
- stage: Defines a particular phase.
- steps: Contains the commands or actions.
- environment: Defines environment variables.
- parameters: Allows users to supply inputs.
- post: Defines actions after pipeline completion.

4. Practical Example

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Terraform Init') {
            steps {
                bat 'terraform init'
            }
        }

        stage('Validate') {
            steps {
                bat 'terraform fmt -check'
                bat 'terraform validate'
            }
        }

        stage('Plan') {
            steps {
                bat 'terraform plan'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully'
        }
        failure {
            echo 'Pipeline failed. Check the console log.'
        }
    }
}
```

This example is intended for a Windows Jenkins agent with Terraform installed and the required AWS credentials and backend configuration available.

5. Scenario-Based Questions

Q: Your Jenkins job works manually but fails when executed through a Jenkinsfile. What do you check?

Answer: I check the Jenkins agent, execution account, PATH, tool installation, credentials, environment variables, workspace, file paths, and permissions. A command may work in my interactive terminal but fail under the Jenkins service account because it uses a different environment.

Q: What is the difference between Declarative and Scripted Pipeline?

Answer: Declarative Pipeline uses a structured syntax with sections such as pipeline, stages, and steps. Scripted Pipeline uses Groovy-based scripting and offers more programming flexibility. I prefer Declarative Pipeline for standard CI/CD workflows because it is easier to read and maintain.

Topic 2: Basic Jenkins Interview Questions

Topic 4: Jenkins Jobs and Builds

1. Definition

A job is a configured task in Jenkins. A build is an individual execution of that job or pipeline.

2. Purpose

Jobs organize automation, while build history helps track execution results and investigate failures.

3. Working

A user or trigger starts a job. Jenkins allocates an executor, runs the configured steps, records logs and results, and marks the execution as successful, unstable, or failed as appropriate.

4. Practical Example

Create a Pipeline job named EKS-Terraform-CI, connect it to your GitHub repository, select the correct branch, and configure it to use the repository's Jenkinsfile.

5. Scenario-Based Questions

Q: A Jenkins job is queued for a long time. What do you check?

Answer: I check executor availability, agent connectivity, the number of queued jobs, whether the job is restricted to a particular label, and whether other builds are consuming all available executors.

Q: What is the difference between a job and a build?

Answer: A job defines the task and its configuration. A build is one execution of that task.

Topic 5: Jenkins Agents, Nodes and Executors

1. Definition

- Controller: Coordinates Jenkins jobs and manages scheduling and configuration.
- Agent: Executes build and deployment tasks.
- Executor: A slot on a node that allows a task to run.

2. Purpose

Agents distribute workloads, isolate environments, and allow different operating systems or tools to be used for different jobs.

3. Working

The controller schedules a pipeline on an eligible node. The node must have an available executor and satisfy any configured labels or tool requirements.

4. Practical Example

Use a Windows agent for a pipeline that runs bat commands, or a Linux agent for a pipeline using Bash and Linux-based deployment tools.

5. Scenario-Based Questions

Q: Jenkins says there are no available executors. What do you do?

Answer: I check the executor configuration, active builds, agent status, queue details, and node labels. If appropriate, I add capacity or optimize the workload rather than blindly increasing executor counts.

Q: Why use agents instead of running everything on the controller?

Answer: Agents separate build workloads from controller responsibilities, improve scalability, and allow jobs to run in suitable environments. For production installations, controller executors are generally restricted for ordinary builds.

Topic 6: GitHub Integration and Webhooks

1. Definition

GitHub integration connects a Jenkins job to source code stored in GitHub. A webhook is an HTTP event notification that can tell Jenkins when a repository changes.

2. Purpose

It enables automated builds when developers push commits or create pull requests, depending on the configured integration.

3. Working

1. Jenkins is configured with the repository URL and branch.
2. Git credentials are configured if required.
3. A webhook or polling trigger detects changes.
4. Jenkins checks out the configured revision.
5. The pipeline runs against that revision.

4. Practical Example

For your e2e-k8s-project1 repository, configure the Jenkins job to check out the master branch and load the Jenkinsfile from the appropriate project directory. Ensure the webhook endpoint is reachable from GitHub if using webhooks.

5. Scenario-Based Questions

Q: A developer pushed code, but Jenkins did not start a build. What do you check?

Answer: I verify the webhook delivery status, Jenkins endpoint accessibility, job trigger configuration, repository URL, branch filters, webhook secret if configured, and Jenkins logs. I also confirm that the commit was pushed to the branch monitored by the job.

Q: Jenkins reports that it cannot find the Jenkinsfile. What is the likely cause?

Answer: The Jenkinsfile may not exist at the configured path or branch. I check the repository, branch, script path, and folder structure. For a Jenkinsfile inside EKS-TF, the script path must point to the correct relative location.

Topic 7: Jenkins Plugins

1. Definition

Plugins extend Jenkins with additional integrations and features.

2. Purpose

They provide support for Git, credentials, cloud platforms, pipeline execution, notifications, test reports, and other automation needs.

3. Working

An administrator installs and configures a compatible plugin. Jenkins jobs then use the plugin's supported steps, integrations, or configuration options.

4. Practical Example

An AWS credentials plugin can expose configured AWS credentials to a pipeline through a credentials binding or supported AWS integration.

5. Scenario-Based Questions

Q: A Jenkins pipeline fails after a plugin upgrade. What do you do?

Answer: I identify the plugin and version involved, review Jenkins logs and compatibility requirements, and check whether dependencies or other plugins changed. I use a tested rollback or compatible version if needed, following the team's change-management procedure.

Q: Should we install every plugin that may be useful?

Answer: No. I install only necessary, trusted plugins, keep them updated through a controlled process, and monitor compatibility and security advisories.

Topic 3: Scenario-Based Jenkins Questions

Topic 8: Git Checkout Failure

1. Definition

A checkout failure occurs when Jenkins cannot retrieve the required source code from the configured repository.

2. Purpose

Troubleshooting checkout failures restores the pipeline's ability to use the correct version of source code.

3. Working

Check the console log for errors such as authentication failure, repository not found, branch not found, network timeout, or Git executable errors.

4. Practical Example

```
stage('Checkout') {
    steps {
        git branch: 'master',
            credentialsId: 'github-repo-credentials',
            url: 'https://github.com/Manjunathamanjuu/e2e-k8s-project1.git'
    }
}
```

Use a credential ID only if the repository requires authentication. For a public repository, the credentials argument may not be needed.

5. Scenario-Based Questions

Q: Jenkins reports Authentication failed during checkout. How do you fix it?

Answer: I verify the repository URL, token validity, repository permissions, and required token scopes. I store the token in Jenkins Credentials and reference its credential ID rather than placing it directly in the Jenkinsfile.

Q: Jenkins reports Couldn't find any revision to build. What does it mean?

Answer: Jenkins may be configured for a nonexistent branch, the branch name may be incorrect, or the repository may not contain the expected commit. I verify the remote branches and the job's branch specification.

Topic 9: Terraform Pipeline Failure

1. Definition

A Terraform pipeline failure occurs when an infrastructure automation stage fails during initialization, validation, planning, or applying changes.

2. Purpose

Troubleshooting Terraform failures ensures infrastructure changes are correct, authorized, and applied safely.

3. Working

Investigate the failed stage in order:

1. terraform init — backend and provider initialization.
2. terraform fmt -check — formatting.
3. terraform validate — configuration validity.
4. terraform plan — intended infrastructure changes.
5. terraform apply — execution of the changes.

4. Practical Example

```
stage('Terraform Validation') {
    steps {
        bat 'terraform fmt -check'
        bat 'terraform validate'
    }
}

stage('Terraform Plan') {
    steps {
        bat 'terraform plan -out=tfplan'
    }
}
```

For a real deployment pipeline, use a consistent backend and workspace, provide the required variables, secure AWS authentication, and ensure that the plan is applied in the correct environment.

5. Scenario-Based Questions

Q: Terraform works locally but fails in Jenkins. What do you check?

Answer: I compare Terraform versions, provider lock files, environment variables, AWS identity, backend configuration, variable files, workspace, working directory, network access, and service-account permissions. I also verify whether Jenkins is using the correct state backend.

Q: The Terraform plan wants to recreate a production resource unexpectedly. What do you do?

Answer: I stop before applying. I inspect the plan, compare the configuration and state, review recent changes and resource attributes, and identify the cause. I do not approve destructive changes until they are understood and authorized.

Topic 10: AWS Credentials and IAM Permissions

1. Definition

AWS credentials identify the principal making AWS API requests. IAM permissions determine which actions that principal can perform.

2. Purpose

Secure AWS authentication allows Jenkins to provision infrastructure without exposing long-lived secrets or granting excessive permissions.

3. Working

Jenkins obtains credentials through an approved mechanism, such as an IAM role on an appropriately configured AWS-hosted agent or a securely stored credential. AWS evaluates the identity and permissions for each API request.

4. Practical Example

```
withCredentials([[
    $class: 'AmazonWebServicesCredentialsBinding',
    credentialsId: 'aws-jenkins-credentials'
]]) {
    bat 'aws sts get-caller-identity'
}
```

This is an illustrative example for an installation where the relevant AWS Credentials plugin and binding are configured. The exact binding depends on the installed plugin and authentication method.

5. Scenario-Based Questions

Q: Jenkins reports AccessDenied when creating an EC2 instance. What do you check?

Answer: I verify the active AWS identity using aws sts get-caller-identity, then review IAM permissions, permissions boundaries, service control policies, resource policies, and any applicable region or account restrictions. I request only the permissions required for the deployment.

Q: How do you secure AWS credentials in Jenkins?

Answer: I store secrets in Jenkins Credentials or an approved secrets manager, restrict access by folder and job, avoid printing secrets, rotate credentials, and prefer temporary role-based credentials where practical. I never hardcode access keys in source code.

Topic 11: Failed Build and Rollback

1. Definition

A failed build means that one or more required pipeline steps did not complete successfully. A rollback restores a service or deployment to a known-good state when a release causes problems.

2. Purpose

Failure handling prevents unsuccessful releases from proceeding and reduces the impact of production incidents.

3. Working

The pipeline stops when a required step fails. Engineers review logs, identify the root cause, correct the issue, and rerun appropriate tests. If a production deployment is unhealthy, the team follows a tested rollback or recovery plan.

4. Practical Example

For an application deployed to Kubernetes, a team may use a supported deployment strategy and retain the previous known-good image. If the new release fails health checks, the team can roll back to the previous revision when the deployment history and strategy support it.

5. Scenario-Based Questions

Q: A deployment succeeded in Jenkins, but users cannot access the application. What do you do?

Answer: I treat this as a production incident. I check application health, logs, service endpoints, Kubernetes pod status, readiness probes, ingress or load balancer configuration, security groups, DNS, and monitoring alerts. If the release caused the outage, I initiate the approved rollback or recovery procedure.

Q: Should Jenkins automatically retry every failed stage?

Answer: No. Retries are useful for transient failures such as temporary network issues. They are not a solution for deterministic problems such as invalid configuration, missing permissions, or failed tests. Retry policies should be limited and appropriate to the operation.

Topic 4: Real-Time Production Issues

Topic 12: Jenkins Disk Space and Workspace Problems

1. Definition

Disk-space issues occur when Jenkins workspaces, build logs, artifacts, caches, or old builds consume available storage.

2. Purpose

Disk management prevents build failures and keeps Jenkins stable.

3. Working

Monitor disk usage, identify large directories, configure build retention and workspace cleanup, and archive only the artifacts that need to be retained.

4. Practical Example

```
pipeline {
    agent any

    options {
        buildDiscarder(logRotator(
            numToKeepStr: '20'
        ))
    }

    stages {
        stage('Build') {
            steps {
                echo 'Run build steps here'
            }
        }
    }

    post {
        always {
            cleanWs()
        }
    }
}
```

This example requires the Workspace Cleanup plugin for cleanWs(). Do not clean a workspace before preserving required artifacts or diagnostic evidence.

5. Scenario-Based Questions

Q: Builds are failing with No space left on device. What is your response?

Answer: I check filesystem usage and identify whether workspaces, logs, artifacts, caches, or temporary files are consuming storage. I remove only safe-to-delete data, apply retention policies, and increase capacity if necessary. I also investigate why storage usage increased.

Topic 13: Jenkins Is Unavailable

1. Definition

Jenkins unavailability means users or automation cannot access the controller or the required service is not functioning correctly.

2. Purpose

Rapid diagnosis restores CI/CD capability and prevents prolonged delivery interruptions.

3. Working

Check service status, application logs, memory, CPU, disk space, Java compatibility, network connectivity, and recent configuration or plugin changes.

4. Practical Example

On a Windows Jenkins server, an administrator can inspect the Jenkins Windows service and its logs, then determine whether the service stopped, ran out of memory, or encountered a startup error.

5. Scenario-Based Questions

Q: Jenkins is not opening at http://localhost:8180. What do you do?

Answer: I check whether the Jenkins Windows service is running, confirm that it is configured to use port 8180, inspect service and application logs, verify whether another process occupies the port, and check local firewall or proxy settings. I avoid restarting repeatedly without reviewing the failure evidence.

Q: Jenkins is running, but every build is slow. What do you check?

Answer: I check CPU and memory usage, disk I/O, executor availability, agent connectivity, queue length, network latency, dependency downloads, and build-specific timings. I use those measurements to identify the bottleneck before changing capacity or configuration.

Topic 14: Production Deployment Failure

1. Definition

A production deployment failure occurs when a release does not deploy correctly or the service becomes unhealthy after deployment.

2. Purpose

A controlled response limits user impact, preserves evidence, and restores service safely.

3. Working

1. Confirm the impact and severity.
2. Stop further rollout if appropriate.
3. Review Jenkins logs and deployment events.
4. Check application and infrastructure health.
5. Roll back or apply a corrective fix using the approved procedure.
6. Verify recovery and communicate the status.
7. Document the root cause and preventive actions.

4. Practical Example

If a new Docker image causes Kubernetes pods to enter CrashLoopBackOff, inspect pod events and logs, verify the image and configuration, and restore the previous known-good version if needed.

```
kubectl get pods -n production
kubectl describe pod <pod-name> -n production
kubectl logs <pod-name> -n production --previous
kubectl rollout status deployment/<deployment-name> -n production
```

The --previous option is useful when a container restarted and logs from its previous instance are available. Use commands only against the intended cluster and namespace.

5. Scenario-Based Questions

Q: A production deployment caused an outage. How do you handle it?

Answer: I prioritize restoring service, notify the incident team, stop further deployment if appropriate, and evaluate rollback options. After recovery, I investigate the root cause, identify gaps in testing or monitoring, and implement preventive actions.

Q: What would you include in a post-incident review?

Answer: I would document the timeline, impact, detection method, root cause, resolution, communication, and corrective actions. The review should focus on improving the system and process rather than assigning blame.

Topic 5: Real-Time Project Explanation

Topic 15: Explain Your Jenkins CI/CD Project in an Interview

1. Definition

A CI/CD project uses automation to move code or infrastructure configuration from source control through validation and testing into a target environment.

2. Purpose

The objective is to deliver changes reliably, reduce manual work, and enforce security and quality checks.

3. Working

For an AWS Terraform and EKS project, the workflow can be:

- **GitHub** — Terraform configuration, manifests and Jenkinsfile
- **Jenkins** — Checkout, validate, test and plan
- **Approval and secure authentication** — Review changes and authorize deployment
- **Terraform and AWS** — Provision or update infrastructure
- **EKS and application deployment** — Deploy, verify health and monitor

4. Practical Example — Interview Project Answer

Use the following as a model answer. Adjust it to match the stages you have actually implemented and tested.

I worked on an AWS infrastructure automation project using Terraform, Jenkins, GitHub, and Kubernetes.

I stored the Terraform configuration and Jenkinsfile in GitHub. Jenkins checks out the required branch and executes the pipeline stages.

The pipeline initializes Terraform, checks formatting, validates the configuration, and generates a plan to show the proposed infrastructure changes. AWS authentication is handled through configured Jenkins credentials, and infrastructure changes are reviewed before applying them.

For the Kubernetes portion, the project uses Amazon EKS. The deployment workflow can be extended to build a Docker image, push it to Amazon ECR, and deploy the application to EKS. After deployment, I verify the deployment status, pod health, logs, and application accessibility.

I focus on secure credential handling, reusable Terraform configuration, clear pipeline stages, troubleshooting from console logs, and controlled changes to infrastructure.

5. Scenario-Based Questions

Q: Explain your project architecture.

Answer: GitHub stores the source code and infrastructure configuration. Jenkins automates checkout, validation, and deployment stages. Terraform provisions AWS infrastructure using a remote state backend where configured. For the Kubernetes workload, Amazon EKS runs the application, and monitoring and health checks help verify the deployment.

Q: What was the biggest challenge in your project?

Answer: One challenge was making Terraform work consistently in both the local environment and Jenkins. I compared tool versions, AWS identity, environment variables, backend configuration, variable files, and the Jenkins workspace. I used the pipeline logs to isolate the issue and validated each stage before proceeding.

Q: How do you protect production from accidental Terraform changes?

Answer: I review the Terraform plan, restrict who can approve and apply changes, secure the state backend and its access, use appropriate IAM permissions, and require approval for high-impact production changes. I also investigate any unexpected resource replacement before applying the plan.

Q: What happens if Jenkins fails after Terraform creates some resources?

Answer: I check the Terraform state and the actual AWS resources before retrying. Terraform operations can create some resources before another operation fails. I inspect the error and state, correct the root cause, and generate a fresh plan. I avoid deleting state files or blindly repeating destructive actions.

Topic 6: Quick Interview Revision

Use these questions to test yourself. The correct answer is marked after each question.

1. What is the main purpose of a Jenkinsfile?

A. To store AWS access keys
B. **To define a pipeline as code** **(correct)**
C. To replace GitHub

2. What should you do if Terraform plans to destroy a production resource unexpectedly?

A. Apply immediately
B. Delete the state file
C. **Stop and investigate the plan** **(correct)**

3. Where should Jenkins secrets be stored?

A. Directly in the Jenkinsfile
B. **In Jenkins Credentials or an approved secrets manager** **(correct)**
C. In public GitHub variables

4. What is the first step when a deployment causes an outage?

A. **Prioritize impact assessment and service recovery** **(correct)**
B. Delete the Jenkins job
C. Upgrade every plugin

5. What does a Jenkins executor represent?

A. A Git branch
B. **A slot for executing a task on a node** **(correct)**

C. An AWS IAM policy

Final Interview Tips

- Explain the concept first, then its purpose, working, and a real example.
- For production scenarios, describe impact -> investigation -> action -> verification -> prevention.
- Be prepared to explain every stage of your own Jenkinsfile.
- Never claim a deployment, monitoring integration, or rollback was implemented unless you actually tested it.
- For AWS and Terraform, understand IAM, remote state, locking, plans, approvals, and recovery.
- For Jenkins, know credentials, agents, executors, plugins, webhooks, logs, and pipeline syntax.
- Practice explaining your project aloud in two minutes, then answer follow-up questions with specific technical details.